

Cours 420-4B6-LI Développement d'application Web I Hiver 2026 Cégep Limoilou Département d'informatique Professeur : Martin Simoneau	TP1 (10 %) – Loterie C#
--	---

Objectifs

- Utiliser le langage C# pour programmer une application en console qui tire des numéros de loto et évalue les gains.

Consignes :

- Remettre vos projets sur Omnivox/Léa à la date indiquée.
- Le travail se fait en équipe de 2 déterminée par l'enseignant. *Github* doit être utilisé.
 - Commitez votre propre code. Le code est automatiquement attribué à celui qui le commit. Si vous travaillez à plusieurs sur un bout de code, mettez tous les noms au début du message de commit.
 - Ne pas commitez le code des autres. Vous perdrez des points si vous le faites !
- La copie est interdite. Tout code provenant d'une aide extérieure (ex.: ami, famille, *StackOverflow*, IA...) qui n'est pas mentionnée directement dans le code est considéré comme de la triche et occasionnera la note 0 pour le TP1. Vous n'aurez aucun point pour les parties faites par une aide extérieure. Tout code remis peut faire l'objet d'une interrogation en profondeur afin de s'assurer que vous en êtes bien l'auteur. On peut aller jusqu'à vous interroger sur les technologies que vous avez utilisées.
- Surveillez le canal TP1 sur Teams pour des mises à jour ou des modifications de l'énoncé.

Remises :

Remettre le projet complet dans une archive ZIP sur LÉA à la date indiquée dans LÉA.

À faire

• Contexte :

On veut créer une application qui simule des tirages de loterie. L'application aidera à déterminer les chances de gagner et les gains réalisables. Il y a deux modes de fonctionnement:

- Effectue un nombre de tirages prédéterminé
- Effectue des tirages jusqu'à l'atteinte d'une condition préétablie.

• Description du système

- Les billets de loterie
 - Il existe 2 types de billets:
 - **Combinaison** : l'ordre n'a pas d'importance et les numéros sont tirés une seule fois (tirage de boule sans remise).
 - **Séquence** : L'ordre est important et les numéros peuvent être tirés plusieurs fois (tirage avec remise).
 - Peu importe le type, un billet contient toujours **6 numéros**.
 - Les numéros vont de **0** jusqu'à une **valeur maximum** (qui peut être configurée).
 - Un billet une méthode pour :
 - Comparer 2 billets (un billet représente le tirage et l'autre le billet du client) et déterminer le nombre de numéros gagnants:
 - **Séquence** : la position des numéros importe et ils doivent former une séquence de 3, 4, 5 ou 6. Les numéros de la séquence doivent être les uns à côté des autres.
 - Par exemple : avec le tirage(2,1,3,4,5,6) et le billet(2,1,6,4,5,8) il y a une séquence max de 2 (1,2 ou 4,5)
 - Avec le tirage(1,2,3,6,4,5) et le billet(1,7,3,6,4,5), il y a 4 numéros gagnants en séquence (3,4,5,6).
 - **Combinaison** : la position des numéros n'importe pas.
 - Les utilisateurs (optionel):

- La gestion des utilisateurs n'est pas obligatoire, ils font partie des tâches au choix (voir plus loin).
- Il n'y a qu'un seul utilisateur Invité/guest.
- Il n'y a pas de login, il suffit de changer d'utilisateur avec le handler
- Il y a 3 types d'utilisateurs:
 - Admin : il peut:
 - Se connecter
 - créer des utilisateurs;
 - réinitialiser les données;
 - lancer les tirages/simulations;
 - Guest: il peut:
 - se connecter;
 - voir les tirages
 - Client: il peut:
 - Se connecter
 - Acheter des billets
 - Voir ses résultats (montants qu'il a gagné)
- Le système doit fonctionner autour d'un patron de **Chain of Responsibility**. Vous pouvez découpez les responsabilités à votre goût, mais on vous donne les repères suivants pour vous aider :
 - Tous les handlers reçoivent un dictionnaire avec des chaîne de caractères en clé un objet en valeur. Chaque Handler pourra au besoin consulter les valeurs déjà présentes et y ajouter d'autres valeur suite à son travail. Des valeurs typiques sont:
 - Les configurations :
 - Valeur max à considérer
 - Le nombre de tirage à faire
 - Le type de billets tirés
 - Le tableau des numéros tirés
 - Le(s) billet(s) de référence de(s) l'utilisateur(s)
 - Les montants gagnés
 - Les handlers ne doivent effectuer qu'une seule tâche afin que la chaîne soit configurable et réutilisable.
 - Les handlers à ne pas manquer sont:
 - Installation des paramètres par défaut dans le dictionnaire de valeur
 - Tirage des numéros gagnants (nombre de tirages fixe). Concrètement, tirer un numéro gagnant revient simplement à créer un billet avec un numéro au hasard.
 - L'évaluation des billets gagnants. Regroupement par nombre de numéros gagnant
 - Calcul des montants gagnés
 - Saisi des paramètres (en console ou directement dans le code)
 - Affichage final
- Tâches au choix. Une équipe de 2 doit produire 100 points et une équipe de 3 doit produire 135 points. Les points excédentaires d'une personne lui sont attribués personnellement et non à l'équipe.
 - Créer les classes pour le patron de COR (10pt) (obligatoire)
 - Créer 2 scénarios complets qui utilisent les handlers (obligatoire)
 - Un passif sans interaction en console (5pts)
 - Un interactif avec décision de l'utilisateur (10pts)
 - Calculer des montants gagnés et statistiques (10 pts)(obligatoire)
 - Établissez vous-même les montant à gagner pour que votre application soit "rentable" et intéressante !
 - Montant total gagné
 - Montant total pour chaque nombre de numéros gagnants (la somme de tous les billets à 3 numéros, la somme de tous les billets à 4 numéros...).
 - Montant total dépensé par type de billet.
 - Si vous gérer des utilisateurs, les statistiques sont gérées par utilisateurs (+5pts) (optionnel).
 - Programmer un billet séquence avancée avec séquence n'importe où (10pts)
 - Modifier l'algorithme de séquence pour que chaque séquence trouvée dans le billet tirée puisse être valide peu importe où elle se trouve dans les billets de référence.
 - Simuler avec tirage "jusqu'à" (30 pts)

- Modifiez l'architecture du COR pour lancer une tâche à répétition jusqu'à l'atteinte d'une ou plusieurs conditions. La condition est saisie par un delegate dans le code.
- Calculer des montants gagnés et statistiques (10 pts)
 - Montant total gagné
 - Montant total pour chaque nombre de numéros gagnants (la somme de tous les billets à 3 numéros, la somme de tous les billets à 4 numéros...)
 - Si vous avez des utilisateurs, les statistiques sont gérées par utilisateurs (+5pts)
- Créer le billet de type combinaison (20 pts)
- Créer le billet de type séquence (25 pts)
- Créer les 3 types d'utilisateurs et de leur droit (25 pts)
 - Gérer plusieurs billets de référence à comparer avec les tirages
 - Gérer le changement d'utilisateur
 - Gérer l'association des tâches
- Autres (faite obligatoirement valider par l'enseignant)
- Contraintes
 - Chaque personne devra:
 - Montrer les acquis des éléments vus dans le cours :
 - Delegate / interface (callback)
 - *Action*
 - *Func*
 - *Predicate*
 - Nullable
 - Propriétés C#
 - Tests unitaires NUnit
 - Collection (*List*, *Hashset* et *Dictionary*)
 - Tuple / valeur par défaut / yield return / as / is
 - Travail d'équipe
 - Réserver une tâche et donner une date de fin. Tant que la date de fin prévue n'est pas arrivée personne ne peut la faire à votre place. Si la date de fin prévue arrive un autre peut prendre votre tâche et la faire à votre place sans vous demander votre avis. Essayez de communiquer efficacement si vous avez des difficultés. Utilisez le canal Teams pour des échanges officiels. Par exemple, si un coéquipier ne vous donne pas de nouvelles, passez par votre canal d'équipe pour que le professeur puisse intervenir.

Critères d'évaluation et exigences:

Exigences

- Mettre vos noms en commentaires dans l'en-tête de chacune des classes dans lequel vous avez travaillé.
- Respectez la date de remise.
- Toutes les méthodes avec des contraintes ou avec des traitements doivent faire l'objet d'un test unitaire et posséder une documentation en bonne et due forme.
- Respectez la programmation par contrat (documentation de classe et de méthode, validation des paramètres et tests unitaires).
- Commitez votre propre code. Le code est automatiquement attribué à celui qui le commit. Si vous travaillez à plusieurs sur un bout de code, mettez tous les noms au début du message de commit. Ne pas commitez le code des autres. Vous perdrez des points si vous le faites !
- Tests unitaires *NUnit*
 - Testez bien toutes vos classes
 - Condition des paramètres
 - Valeurs limites dans les algos
 - Jeux d'essais
 - Vous pouvez vous aider de l'IA pour faire générer des tests, mais assurez-vous que vous maîtrisez le code qu'il vous propose.

- Le fonctionnement de votre code sera évalué par vos tests unitaires. S'il n'y a pas de tests, on présume que le code ne fonctionne pas ! Codez vs tests au fur et à mesure que vous avancez dans le programme. Ne surtout pas attendre à la fin !!!
- Utilisez toujours la visibilité la plus fermée possible.
- Validation de valeur pour setter de propriétés (programmation par contrat)

Qualité du code

- Évitez les type *var*. Faites l'effort de mettre vous-même le véritable type.
- **Commentaires** pertinents et suffisants
- Les **noms** des méthodes, des classes et des attributs sont pertinents et **complets**.
 - Les noms des interfaces commencent par un "I" (i majuscule).
- Toutes les paramètres des méthodes et toutes les propriétés sont protégés par des validations adéquates.
- Il n'y a **pas** de **code inutile** ou commenté.
- **Formatage** effectué dans chaque classe.
- **Algorithmes simples** et efficaces.
- **Découpage du code**,
 - Maximum de 10 lignes de code par méthode (sans les commentaires)
 - Bien découper les *namespace*
- **SOLID** : respectez les trois premiers principes
 - **Single Responsibility**
 - **Open close principle**
 - **Liskov principle**.
- Le code fonctionne **sans erreurs**.
- **Toutes** les tâches obligatoires demandées ont été accomplies.
- N'utiliser pas les instructions: **continue**, **break** (sauf dans la switch) et **goto**.
- Évitez les instructions return multiples dans des imbrications complexes.
- Classification et polymorphisme
 - La réutilisation et la compatibilité sont sans failles. Respect des 3 premiers principes OO **SOLID** (*Single-responsibility, open-closed, Liskov substitution principle*)
 - La virtualité est utilisée aux bons endroits (*virtual* et *override*).
 - **L'abstraction** est toujours utilisée lorsqu'elle est pertinente (*abstract*).

FIN